

Day 2 · 프로젝트 가이드 워크북

Coding Agent 실전

AI기반 TDD 마스터 과정

관통 프로젝트 — 마방진(Magic Square) 4×4 검증기 · 10선 합 34, 어느 줄이 왜 틀렸는지 10초 안에

세션제 하루 완주

계약 INV·E·UC

Dual-Track ≡ ARRR

Command 체인 6종

황금공식 하네스

7단계 관통 프로젝트 — ①주제 ②설계 ③Ask ④Respond ⑤Refine ⑥Repeat ⑦확장(GUI)

하루 8시간(08:30–17:30) · 강의 40% + 실습 60% · 개발자 및 입문자 대상

2026 · 김대경 Ph.D.

이 과정을 관통하는 하나의 생각

인과 사슬 — 오늘의 모든 기법은 한 곳을 향한다

Mom Test · 계약 · 하네스 · ARRR · 계측 — 결국 “개념(요구·불변식)이 코드까지 의미를 잃지 않고 이어지게 하는 것”, 즉 C2C를 위한 것이다.



한 줄로 새기면 오늘은 성공

Ask = RED **Respond** = GREEN **Refine** = REFACTOR **Repeat** = 다음 계약 ID로 다음 질문

하루 시간표 — 세션제로 완주한다 (08:30–17:30)

과정 팩트시트

핵심 목표

마방진 관통 프로젝트로 C2C를 1일 체득

관통 예제

Magic Square 4×4 — 10선 합 34 검증기

핵심 함수

validate_lines · find_blank_coords

교육 방식

강의 40% + 실습 60% (개념→시연→실습 순환)

도구 스택

VS Code · Gemini CLI · Code Assist · pytest · Git

사전 준비

Node.js 20+ · Python 3.11+ · Git · GitHub·Google

08:30	오리엔테이션	환경 점검 · “우리는 C2C를 한다”
08:55	세션 1	바이브 vs C2C · 3수준 사다리 · 미니 TDD 체험
10:30	세션 2	4계층 · 계약 ID · Dual-Track ≡ ARRR · 문제 정의 · 7단계
13:00	세션 3	Mom Test→계약(L0~L2) · R-G-I-O·PRD·하네스
14:40	세션 4	③~⑦ ARRR — RED·GREEN·Golden·REFACTOR·Repeat·확장
16:15	세션 5	AI 문서화 · /export · 매트릭스 마감 · AARRR 계측
17:10	마무리	KPT 회고 · 현업 적용 액션 아이템

Magic Square 4×4 — 하루를 하나로 잇는 예제

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

모든 행·열·대각선의 합 = 34

검증 라인 10개: R1~R4 · C1~C4 · D1 · D2

도메인

- 4×4 · 채운 칸 1~16 · 빈칸 0 · 마법상수 34
- 검증 대상 10선: R1~R4 · C1~C4 · D1·D2
- 목표 = 풀기 아님 → 어느 줄이 왜 틀렸는지 10초 안에
- 자동 풀이·채우기 = OOS(범위 밖)

`validate_lines(grid) → {status, failed_lines}`

7단계 = 논문 8단계 골격 + 13 STEP의 실행 · ③~⑥이 ARRR = C2C × Dual-Track TDD

무엇을 만드나 — 마방진 검증기 (풀지 않고 검증한다)

Magic Square 4×4

1~16을 한 번씩 배치, 마법상수 34. 빈칸(0)이 있는 “부분 마방진”에서 어느 줄이 왜 틀렸는지 10초 안에 확인.

검증 대상 = 10선 R1~R4 · C1~C4 · D1 (↘) · D2 (↙)
각 선의 합이 모두 34여야 pass

incomplete 빈칸(0)이 하나라도 있음 [INV-3]

pass 16칸 완성 + 10선 합 모두 34 [INV-1]

fail 완성 + 34 아닌 줄 존재 [INV-2] → failed_lines

두 계약 함수: `validate_lines(grid)` · `find_blank_coords(grid)`

7단계 진행 — ③~⑥이 ARRR (곧 C2C × Dual-Track TDD의 실행)

① 주제 선정

Mom Test → 제품 계약(L0~L2)

② 프로젝트 설계

R-G-I-O·PRD·계약 ID·스켈레톤·하네스

③ Ask = RED

계약 ID별 실패 테스트

④ Respond = GREEN

실패 로그 → 최소 구현

⑤ Refine = REFACTOR

Golden Master 위 Safe Refactor

⑥ Repeat

문서화(/export) → 다음 계약

⑦ 기능 확장

GUI(Track A)·계측 등 새 사이클

완료 기준: 골든셋 G1~G5 전부 기대대로 · pytest 전부 GREEN · 매트릭스 빈칸 없음

01

세션 1 · 개념 리마인드 ①

바이브 코딩 vs C2C 기반 코딩

3수준 사다리(프롬프트·컨텍스트·하네스) · 미니 TDD 체험 — “추적 가능한가?”

08:55–10:15

바이브 코딩 vs C2C 기반 코딩

바이브 코딩 (Vibe Coding)

- 프롬프트 → 완성 코드. 빠르지만 역추적 불가
- "정말 요구를 만족하나?" 판단 근거가 없음
- 고칠 때마다 불안 — 바닥은 낮추지만 천장이 없다

C2C 기반 코딩

- 모든 동작이 계약 ID(INV-*/E-*)에서 출발
- "ID 없는 동작은 구현하지 않는다"
- 테스트 ↔ 계약 ↔ 커밋으로 추적 — 천장을 지킨다

핵심 질문의 전환: "어떤 기법을 쓰는가" → "추적 고리가 끊긴 곳이 어디인가"

SESSION 1

AI 협업의 3수준 사다리

① 프롬프트

이번 한 번의 지시

“이 실패 로그로 최소 구현해줘”

② 컨텍스트

지금 상황을 붙여 줌

@src/validate_lines.py · pytest 실패 로그

③ 하네스

영구 규칙·도구로 고정

GEMINI.md · settings.json · /커맨드 · 테스트 루프

위로 갈수록 의도가 1회성 지시 → 영구 구조로 고정 · 반복 규율일수록 하네스로 올려라

02

세션 2 · 개념 리마인드 ②

C2C × Dual-Track TDD ≡ ARRR

4계층 검증 · 계약 ID(추적의 못) · 추적성 매트릭스 · 문자열 덧셈 미니 예제

10:30-11:45

C2C 4계층 검증 구조 — 위에서 아래로

임의의 구현 줄에서 위로 올라가면 반드시 불변식에 닿고, 불변식에서 아래로 내려가면 그것을 검증하는 테스트와 지키는 구현에 닿는다. 이 양방향 도달 가능성이 C2C다.

Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

마방진 대응 INV(10선 합 34·상태 규칙) = Layer 1·4(Entity) · UC(격자 UI) = Layer 2(Boundary) · E(입력 검증) = 경계 계층.

계약 ID와 추적성 매트릭스 — 마방진 목표표

계약 ID 규율

- 1 모든 계약에 ID 부여 (INV-* · E-* · UC-*)
- 2 RED 테스트는 검증할 ID를 이름/주석에 참조
- 3 GREEN 구현 줄에 충족한 ID를 주석으로
- 4 커밋 메시지에도 계약 ID를 적는다
- 5 계약 → 테스트 → 구현 (역순 금지)

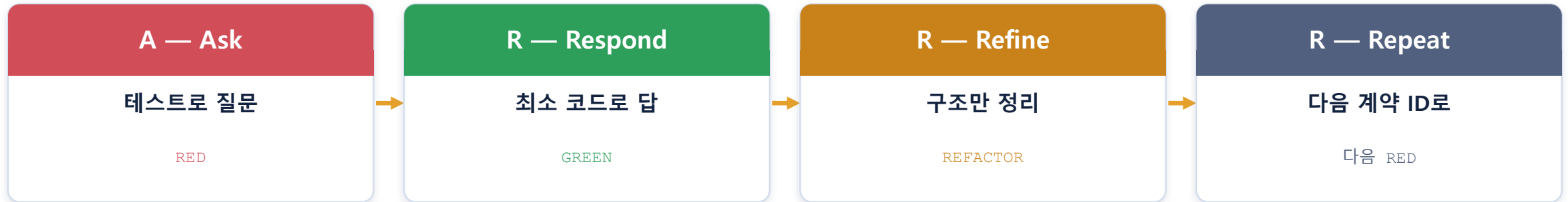
↑ 이 순서를 지키는 한 모든 코드 줄이 계약 ID로 추적된다 — 그것이 C2C.

마방진 검증기 — 확정 계약표(오늘의 SSOT)

ID	계약	증거	계층/트랙	테스트
INV-1	16칸+10선 합 34 → pass	L0	Entity/B	D-VAL-01
INV-2	합≠34 줄 존재 → fail+라인 ID	L0	Entity/B	D-VAL-02
INV-3	빈칸(0) 있으면 incomplete	L1	Entity/B	D-VAL-03
INV-4	failed_lines ⊆ {R·C·D 10선}	L0	Entity/B	D-VAL-04
INV-5	find_blank: 1-index-row-major	L1	Entity/B	D-LOC-01
E-1	grid None → 에러 E003	L0	Bound/A	U-IN-01
E-2	0 아니고 1~16 밖 → E002	L0	Bound/A	U-IN-02
UC-1	격자+[검증]→status-failed 표시	L2	Bound/A	U-GRID-01

OOS(안 할 것): Solver/자동 채우기 · 1~16 중복 검증 — 진짜 문제는 “풀기”가 아니라 “확인”.

Dual-Track TDD ≡ ARRR + 문자열 덧셈 미니 예제



'Dual-Track'은 같은 것의 다른 이름 — ARRR을 돌리는 것이 곧 Dual-Track TDD 실행이며 그것이 C2C를 담보한다.

string_calculator — ARRR 네 박자 실증

```
# A(RED)  assert add("")==0 / add("5")==5 / add("3,5")==8
```

```
# R(GREEN)
```

```
def add(numbers):
```

```
    if numbers == "": return 0
```

```
    return sum(int(n) for n in numbers.split(","))
```

```
# R(Refine) 이름·중복 정리    # R(Repeat) 새 구분자 → 다음 RED
```

각 단계의 완료 신호

RED

의도된 FAIL/ERROR

GREEN

전부 PASS

REFACTOR

계약 불변 · 여전히 GREEN

Repeat

회귀(pytest -q) 통과

03

오리엔테이션 · 환경 준비

도구 3종 · 프로젝트 초기화 · 브랜치 전략

MagicSquare_XX 워크스페이스와 세 형제 폴더 · staging에서 갈라 staging으로 모은다

08:30-08:55

도구 3종의 역할 · 진입점 · 프로젝트 초기화

VS Code

에디터 + 통합 터미널

Gemini CLI

터미널 AI 에이전트 — ARRR 주력

Code Assist

VS Code 확장(채팅·Agent) — 동일 엔진

진입점:

`gemini` 대화형

`gemini -p` 단발

@경로 파일 주입

!cmd 셸 통과

/명령 슬래시

프로젝트 초기화 - MagicSquare_XX + 세 형제 폴더

```
MagicSquare_XX/          # 워크스페이스 (git)
├─ 1_momtest/             # 인터뷰 (상류)
├─ 2_contract/            # 계약의 집
│   └─ docs/PRD.md        # ★ 계약 마스터 (SSOT)
└─ 3_app/                 # 실제 앱 (Delivery)
    ├── GEMINI.md          # 하네스 지도
    ├── .gemini/commands/  # 커스텀 커맨드
    └── src/  tests/  docs/
```

오리엔테이션 체크포인트

- `gemini --version` · `python --version` · `pytest` 실행
- VS Code에 Gemini Code Assist 로그인
- MagicSquare_XX 워크스페이스·가상환경 준비
- 세 형제 폴더 + 3_app 하위(src·tests·docs·gemini)
- GitHub 빈 저장소 · main→staging→spec 브랜치

브랜치 전략 — staging에서 갈라 staging으로 모은다

staging의 형제 브랜치들 (교육용 단계 브랜치)

main

```
└─ staging      # 통합 브랜치
   ├── spec     # 세션3: 계약·PRD·하네스
   ├── red      # 세션4: Ask (실패 테스트)
   ├── green    # 세션4: Respond (최소 구현)
   ├── refactoring # 세션4: Refine
   └─ new_features # 세션4 STEP7: 확장
```

각 브랜치: staging에서 분기 → 작업·커밋 → staging으로 머지

⚠ staging의 의미를 정확히

교육을 위해 의도적으로 실패하는 RED 브랜치도 staging에 머지한 뒤 GREEN을 만든다. 따라서 RED 머지 직후의 staging은 전체 테스트가 실패하며, “언제나 배포 가능한 통합 브랜치”는 아니다. “검증된 것만 모임”은 각 단계 결과가 확인되어 모인다는 교육적 의미다.

커밋 메시지에 계약 ID를 적으면 git log만으로 “어느 계약이 어느 단계에 있는지” 추적된다.

교육용 단계 브랜치 vs 실무 권장 변형

교육·시연 red·green·refactoring 단계 브랜치로 ARRR 흐름을 눈에 보이게. 후 staging 머지.

실무 feature/INV-5-blank-coords처럼 계약 단위 브랜치, TDD 단계는 커밋으로 구분, 전체 GREEN

04

세션 3 · STEP 1 — 상류 Discovery

문제에서 계약으로

Mom Test 3원칙 → 발견→제품 계약(L0~L2) → 확정 계약표·골든셋 → PRD.md 고정

Ask 모드 · 파일 수정 없음

Mom Test 3원칙 — 계약을 발견한다

아이디어에 대한 의견 대신 과거의 구체적 행동·실제 사건만 묻는다. 기능 확인이 아니라 테스트 가능한 계약(INV/E/UC)을 발견하는 Discovery 도구로 쓴다.

①

내 아이디어가 아니라 상대의 삶을 이야기한다

"자동 검증 앱 어때요?" × → "마방진 과제를 어떻게 확인했나요?" ✓

②

의견·미래가 아니라 과거의 구체적 행동을 묻는다

"마지막으로 틀렸을 때 몇 분 썼나요? 어느 줄에서요?" ✓

③

적게 말하고 많이 듣는다

솔루션(TDD·Gemini)을 먼저 팔지 않는다

반드시 물어야 할 것 "마지막으로 틀렸을 때 어떻게 알아챘나?"(→INV) · "잘못된 값 넣었을 때 뭐가 일어났나?"(→E) · "몇 분·몇 줄?"(→증거 L0/L1)

발견 → 제품 계약 · 골든셋 · R-G-I-O · PRD(SSOT)

R-G-I-O — 프로젝트를 한 장으로

Role	부분 마방진 검증 도우미 (풀어 주는 도구 아님)
Goal	“어느 줄이 왜 틀렸는지”를 10초 안에 보여 준다
Input	4×4 list[list[int]] — 0=빈칸, 1~16=채운 칸
Output	{status, failed_lines} + (확장) GUI 표시

골든셋 — Discovery 산출물이자 Delivery 픽스처

G1	빈칸 2개 (2,3)·(4,4) → incomplete
G2	16칸 완성 정상 → pass
G3	(2,2) 6→7 → fail (R2·C2 포함)
G4	grid=None → E003
G5	셀에 99 → E002

PRD.md 고정 (Agent 모드로 전환) — 계약의 마스터 SSOT

2_contract/docs/PRD.md에 [진짜 문제 한 문장 · R-G-I-O · 확정 계약표(INV/E/UC) · OOS · 골든셋 G1~G5 · 성공 기준]을 담는다. 다른 파일은 만들지 않는다. 이어서 README.md에 RED To-Do와 릴리즈 정보를 옮긴다.

✓ **STEP 1 체크** 확정 계약표(INV-1~5·E-1~2·UC-1)+OOS가 PRD.md에 고정 · 골든셋 G1~G5 존재 · 모든 계약에 증거 수준(L0~L2)

Magic Square 계약 확정 — SSOT 계약표

ID	계약	증거	테스트
INV-1	완성+10선 34 → "pass"	L0	D-VAL-01
INV-2	완성+34아닌 줄 → "fail"+라인 ID	L0	D-VAL-02
INV-3	빈칸 있으면 "incomplete"	L1	D-VAL-03
INV-4	failed_lines $\leq$ 10선	L0	D-VAL-04
INV-5	find_blank_coords 1-index row-major	L1	D-LOC-01
E-1 / E-2	None→E003 / 1~16 밖→E002	L0	U-IN-01/02
UC-1	4×4 격자+[검증] 버튼→결과 표시	L2	U-VAL-01

OOS(안 할 것): 자동 풀이·채우기 · 1~16 중복 검증 — 과잉 명세는 계약이 아니라 OOS로

05

세션 3 · STEP 2 — 하네스 구축

Gemini 기술 체계

Harness 골격 · GEMINI.md 금지 4조항 · settings.json · /tdd-red 첫 슬래시 버튼

Agent 모드

GEMINI.md(금지 4조항) · settings.json · /tdd-red

① GEMINI.md — 40~60줄 “지도”

도메인(4×4·34·10선)·API 계약·TDD 순서·ECB를 담고, 변경 후 /memory refresh.

금지 4조항 (절대 규칙)

- 1 테스트 약화 금지 — assert 완화·skip·xfail·수정/삭제
- 2 과잉 구현 금지 — 요청 계약 ID 외 선제 구현
- 3 시그니처 임의 변경 금지 — 바꾸려면 Discovery로
- 4 ID 없는 동작 구현 금지 — 매트릭스에 없으면 정체불명

+ ECB: entity는 boundary/control import 금지, 에러코드는 boundary 소관

② settings.json

승인 모드와 도구 통제. push·rm은 사람이 직접.

default

매번 승인 — 권장 시작점

auto_edit

편집만 자동 — 익숙해진 후

plan

읽기 전용 — 리뷰·스멜 탐지

```
excludeTools:  
  ShellTool(git push)  
  ShellTool(rm)
```

③ /tdd-red — 첫 슬래시 버튼

응답 첫 줄에 Phase 선언. src/ 수정·assert 완화·한 번에 여러 케이스 금지.

첫 RED — 하네스가 살아있는가

```
/tdd-red  
D-VAL-02 [INV-2]:  
완성 격자 (2,2)=6→7  
→ R2·C2 = 35 fail  
기대: status="fail",  
failed_lines ⊇ {R2,C2}  
src/ 수정 금지
```

```
$ pytest -v  
  
test_d_val_02 ... FAILED  
  
← 구현이 없으니 정상(RED)  
스켈레톤: 시그니처+TODO만, 완성 코드는 감춤
```

SESSION 3

금지 4조항 — 켄트 벡의 경고 신호를 규칙으로

1. 테스트 약화 금지 — assert 완화·skip·xfail·수정/삭제 금지
2. 과잉 구현 금지 — 요청한 계약 ID 외 기능 선제 구현 금지
3. 시그니처 임의 변경 금지 — 바꾸려면 Discovery로 돌아가 계약부터
4. ID 없는 동작 구현 금지 — 매트릭스에 없으면 정체불명 코드

마법상수 34 = SSOT(constants.py) · 순서 RED→GREEN→REFACTOR · 첫 줄 Phase 선언

06

세션 4 · STEP 3~6 — Delivery

ARRR 실행

A(RED) → R(GREEN) → Golden Master → R(REFACTOR) → R(Repeat) · 한 번에 계약 하나씩

14:40-16:05

ARRR ↔ 개발 활동 ↔ Gemini 요소 (이 표가 핵심)

ARRR	개발 활동	Gemini 요소(커맨드)
A · Ask (RED)	계약 → 실패 테스트 먼저	<code>/red-test-plan</code> · <code>/red-skeleton</code>
R · Respond (GREEN)	실패 로그 → 최소 구현	<code>/green-minimal</code> · <code>/golden-master</code>
R · Refine (REFACTOR)	동작 불변, 구조만 정리	<code>/refactor-smell</code> · <code>/refactor-safe</code>
R · Repeat Command 체인 6종	문서화 → 다음 계약으로	<code>/export</code> · <code>pytest</code> 회귀

`/red-test-plan`RED 설계표만 (파일 생성 금지)
→ R(GREEN) → Golden Master

`/red-skeleton`pytest.fail 스케레톤만
→ R(GREEN) → Golden Master

`/green-minimal`최소 구현 + 계약 ID 주석
→ R(GREEN) → Golden Master

`/golden-master`출력 계약을 기준 파일로 고정

`/refactor-smell`스멜 탐지만 (수정 금지)

`/refactor-safe`선택한 스멜 1개만

예: D-LOC-01 [INV-5] — 빈칸 좌표 1-index·row-major

A · Ask (RED)

/red-test-plan → /red-skeleton

설계표(계약 인용·G/W/T) → pytest.fail 스케레톤. tests/만, src/ 금지. FAIL 확인

R · Respond (GREEN)

/green-minimal

실패 로그 근거로 최소 구현. 구현 줄에 [INV-5] 주석. 1커밋=1묶음

Golden Master

/golden-master

PASS 확인 후 출력 계약을 기준 파일로 고정 (회귀 기준선)

R · Refine (REFACTOR)

/refactor-smell → /refactor-safe

스멜 탐지만(수정 금지) → 선택한 1개만. 입출력·계약 불변

R · Repeat

/export → 다음 계약

문서화 후 다음 계약 ID로. pytest 회귀 확인

추적성 매트릭스 중간 점검

한 계약을 마칠 때마다 계약 ID ↔ 테스트 ↔ 구현이 이어졌는지 확인한다.

INV-5 D-LOC-01 find_blank ✓

INV-2 D-VAL-02 validate ✓

INV-1 D-VAL-01 validate ✓

빈칸이 없으면 그 계약의 C2C가 성립. 정체불명 코드(계약 ID 주석 없는 줄)는 제거 대상.

Track A GUI — 같은 계약 위에 얹은 UI를 올린다

검증 로직(Track B)이 계약대로 완성되면, 같은 계약 위에 UI(Track A)를 올린다. new_features 브랜치를 staging에서 가른다.

UC-1 — 격자 + [검증] 버튼

4x4 격자와 [검증] 버튼을 표시하고, 눌렀을 때 status.failed_lines를 화면에 보여 준다. (테스트: U-GRID-01 · U-VAL-01)

```
/red-test-plan Phase: red | Layer: boundary | Track: UI
목록: U-IN-01[E-1], U-IN-02[E-2] · validate_lines는 mock
```

Track A의 규율 (ECB)

- ✓ boundary 계층에서만 에러 코드(E002·E003) 처리 — entity는 에러 코드를 모른다
- ✓ 도메인(validate_lines)을 mock으로 격리 — UI 계약(U-GRID-01·U-VAL-01)만 테스트
- ✓ entity → boundary import 금지 (방향 단속)

```
$ git switch staging && git switch -c new_features # 확장 작업 커밋 후
$ git switch staging && git merge --no-ff new_features
```

핵심 Track B(도메인)와 Track A(UI)는 같은 계약을 공유하되 계층을 분리한다. UI는 도메인을 mock으로 격리해 UI 계약만 검증하므로, 도메인이 바뀌어도 UI 테스트는 안정적이다.

07

심화 · 하네스 확장편

Skill · Sub-Agent · Hooks

팀·프로젝트 규모가 커질 때 하네스를 어디까지 키울 수 있는가 — AI 개발 황금 공식

세션 3~4의 Rule·Command로도 하루 완주 가능

Skill · Sub-Agent · Hooks — 하네스를 키운다

Skill

“어떻게 실행하는가”의 절차서

.gemini/skills/<이름>/SKILL.md · 트리거 조건이 맞을 때만 꺼내 쓴다

예: magic-square-tdd · magic-square-docs

Sub-Agent

생성과 검증의 분리

생성 Agent와 검증 Agent를 나눈다. 모두 분석·가이드 전용(수정은 본 Agent)

예: tdd-coach · contract-auditor · qa-tester · logic-reviewer

Hooks

이벤트에 거는 자동 안전장치

Agent 루프의 정해진 이벤트(세션 시작/종료·도구 사용 전후)에 강제 실행

예: guard-destructive-git · 세션 종료 시 pytest 회귀

AI 개발 워크플로우 황금 공식

하네스 = **Rule** (무엇을) + **Command** (반복 버튼) + **Skill** (절차) + **Sub-Agent** (검증 분리) + **Hooks** (강제 안전장치)

그 하네스 위에서 계약(INV/E/UC)을 ARRR로 코드화하면 — 개념이 코드까지 추적되는 C2C가 완성된다.

08

세션 5 · 문서화 & 하류 귀환

개념의 귀환 · 사이클 닫기

AI 문서화 3원칙 · 작업 보고서 8섹션 · /export · AARRR 계측 · KPT 회고

16:15-17:30

AI 문서화 3원칙 · 보고서 8섹션 · AARRR 계측

AI 문서화 3원칙

1 SSOT를 인용

docs/PRD.md·GEMINI.md·git log를 인용할 뿐 새 사실을 만들지 않는다

2 추적 가능

모든 서술을 계약 ID·커밋·테스트로 되짚게 쓴다

3 자동 생성

/export로 보고서·대화록을 자동 번호(NN)로 남긴다

작업 보고서 8섹션 (Report/NN.REPORT.md)

- | | |
|------------------|---------------|
| ① 문제·목표(R-G-I-O) | ② 확정 계약표 |
| ③ 골든셋 | ④ AARRR 진행 로그 |
| ⑤ 추적성 매트릭스 | ⑥ 리팩터 기록 |
| ⑦ 남은 OOS·다음 후보 | ⑧ KPT 회고 |

AARRR 계측 — 개념의 귀환

Activation="검증 버튼을 눌러 결과를 본 첫 순간",
Retention="다른 판으로 재검증"처럼 재정의하고,
계측 이벤트를 계약(M-1·M-2)으로 코드화한다(계측 모듈도 Boundary에서 AARRR).

계측도 AARRR로 개발

폐회로가 닫힌다 — C2C의 완성형

진짜 문제(Mom Test)



제품 계약(INV/E/UC)



하네스



AARRR(R→G→R)



추적성 매트릭스



문서화·계측

실측이 목표에 미달하면 그 수치가 다음 Discovery(Mom Test)의 질문이 된다 — 개념→코드→실측→다음 개념.

문서화 — /export로 세션 기록 남기기

AI 문서화 3원칙

- SSOT 인용 — PRD·GEMINI.md·git log만
- 추적 가능 — 계약 ID·커밋·테스트로
- 자동 생성 — /export → Report/NN·Prompting/NN

작업 보고서 8섹션

- 문제·목표(R-G-I-O) · 계약표 · 골든셋
- ARRR 로그 · 추적성 매트릭스 · 리팩터
- 남은 OOS·다음 후보 · KPT 회고

문서는 새 사실을 만들지 않는다 — SSOT를 인용해 추적 가능하게 남긴다

KPT 회고 — 다음 회전의 가설 만들기 (STEP 13)

Keep

계속할 것

계약→테스트→구현 순서 · 한 번
에 하나씩 · 커밋에 계약 ID

Problem

문제

막힌 지점·비용이 큰 규율·도구 마
찰

Try

시도

다음 사이클에 바꿀 실험 1~2개

회고는 감상이 아니라 다음 Discovery의 입력 — Try가 다음 사이클의 가설이 된다

마방진 하나로 한 바퀴를 완주했다

진짜 문제(Mom Test) → 제품 계약(INV/E/UC) → 하네스 → ARRR → 추적성 매트릭스 → 문서화. 개념이 코드로 내려갔다가 문서로 닫히는 한 바퀴.

Keep

계약→테스트→구현 순서 · 한 번에 계약 하나씩 · 커밋에 계약 ID

Problem

막혔던 점을 적는다 (경계값·ECB 방향·과잉 구현 등)

Try

다음에 시도할 것 (Skill·Sub-Agent·Hooks·plan 모드 Review)

현업 적용 — 팀 액션 아이템

- 우리 팀의 '진짜 문제'를 Mom Test로 다시 확인한다 (솔루션 먼저 말하지 않기)
- 핵심 규칙을 GEMINI.md(또는 AGENTS.md)로 고정하고 금지 4조항을 팀 표준으로
- 모든 PR을 계약 ID ↔ 테스트 ↔ 커밋으로 추적 가능하게 만든다

마 무 리

개발자 역할의 재정의

코드의 작성자에서, C2C 추적성의 설계자·감리자로.

개발자의 명시적 산출물은 계약 표, 골든셋, 실패하는 테스트, 규칙 파일, 프롬프트다.
구현 코드의 상당 부분은 AI가 생성하되 — 계약 ID·테스트·규칙이 그 품질을 게이팅한다.
AI에 위임하되 주도권을 쥐고 씨앗을 먹지 않는 것, 그것이 Augmented Coding이다.

C2C

Dual-Track TDD

ARRR

하네스

AARRR